



GESTION DE BASE DE DATOS

NOMBRES:

MENDOZA DEGADO RAMÓN

BAILON HERRERA ELIZABETH

Docente:

Ing. John Cevallos Macías, Mg.

Ciudad:

Manta – Ecuador

2026-1

1.INTRODUCCIÓN	5
2.HITO 1.....	6
Descubrimiento del problema	6
2.1. Nombre del proyecto	6
2.2. Descripción del problema.....	6
2.3. Objetivos del proyecto	7
2.3.1. Objetivo general.....	7
2.3.2. Objetivos específicos	7
2.4. Usuarios del sistema	7
Cliente	7
Administrador del taller	7
2.5. Investigación realizada	8
2.6. Entrevistas realizadas	8
Entrevista 1.....	8
Respuesta destacada.....	8
Hallazgos.....	8
Entrevista 2.....	9
Respuesta destacada.....	9
Hallazgos.....	9
2.6.1. Entrevista en video.....	9
2.7. Análisis de soluciones existentes.....	10
2.8. Propuesta de solución	10
2.9. Arquitectura general del sistema.....	11
2.10. Módulos implementados	11
2.10.1. Módulo de Gestión de Turnos	12
2.10.2. Módulo de Servicios Técnicos	12

2.11. Entidades del sistema	12
2.12. Regla de negocio principal	13
2.13. Conclusión del Hito 1	13
3. HITO 2	13
Desarrollo de la API REST	13
3.1. Introducción	13
3.2. Tecnologías utilizadas	14
3.3. Organización del proyecto	14
3.4. Arquitectura implementada.....	15
3.5. Módulos desarrollados	16
3.5.1. Módulo de Gestión de Turnos.....	16
3.5.2. Módulo de Servicios Técnicos	16
3.6. Modelos implementados	17
3.7. Operaciones CRUD	17
Gestión de Turnos.....	17
Servicios Técnicos	18
3.8. Validaciones implementadas.....	18
3.9. Pruebas realizadas	19
3.10. Resultados obtenidos	19
3.11. Conclusiones del Hito 2.....	19
4. HITO 3	20
Integración, seguridad y despliegue del sistema	20
4.1. Introducción	20
4.2. Integración de la base de datos.....	20
4.3. Modelado de la base de datos	20
4.4. Autenticación mediante JWT	21
4.5. Middleware de autenticación.....	22

4.6. Organización de las rutas	22
4.7. Contenerización mediante Docker.....	23
4.8. Docker Compose.....	23
4.9. Integración continua mediante GitHub Actions	23
4.10. Flujo general del sistema	24
4.11. Resultados obtenidos	25
5. Conclusiones generales	25
6. Recomendaciones.....	26

1. INTRODUCCIÓN

Actualmente muchos talleres mecánicos pequeños de la ciudad de Manta todavía administran sus turnos y servicios mediante llamadas telefónicas, mensajes de WhatsApp o registros escritos en cuadernos. Aunque estos métodos permiten llevar un control básico de las actividades diarias, presentan limitaciones cuando aumenta la cantidad de clientes o se requiere consultar información sobre trabajos realizados anteriormente.

La falta de un sistema centralizado ocasiona retrasos en la atención, dificultades para organizar la carga de trabajo y problemas al momento de consultar el historial de los vehículos. Además, tanto los propietarios de los talleres como los clientes dependen de la comunicación constante para conocer el estado de un servicio o confirmar la disponibilidad de un horario.

Con el propósito de aportar una solución a esta problemática, se desarrolló una API REST utilizando el lenguaje de programación Go. El sistema permite administrar la información relacionada con los turnos y los servicios técnicos, incorporando mecanismos de autenticación, persistencia de datos y una arquitectura organizada que facilita el mantenimiento y la escalabilidad del proyecto.

Durante el desarrollo se aplicaron los conocimientos adquiridos en la asignatura Aplicaciones Web II, empleando herramientas como Chi Router para el manejo de rutas, GORM como ORM, PostgreSQL para el almacenamiento de la información, JWT para la autenticación de usuarios y Docker para facilitar la ejecución del proyecto en distintos entornos.

El presente documento describe las diferentes etapas del desarrollo del sistema, desde la identificación del problema hasta la implementación de los principales componentes de la aplicación, mostrando las decisiones tomadas durante el proceso y los resultados obtenidos.

2. HITO 1

Descubrimiento del problema

2.1. Nombre del proyecto

Sistema web para la gestión de turnos y servicios técnicos en talleres mecánicos de la ciudad de Manta.

2.2. Descripción del problema

En la ciudad de Manta existen numerosos talleres mecánicos pequeños que continúan administrando sus actividades mediante herramientas tradicionales como cuadernos, hojas de cálculo, llamadas telefónicas y conversaciones por WhatsApp. Aunque estos mecanismos resultan suficientes cuando el número de clientes es reducido, presentan dificultades a medida que aumenta la demanda de servicios.

Durante las visitas realizadas a distintos talleres se observó que la asignación de turnos depende principalmente de la disponibilidad que recuerda el encargado o de anotaciones manuales. Esto provoca retrasos, pérdida de información y dificultades para conocer el estado real de cada reparación.

Entre los principales inconvenientes detectados se encuentran:

- Registro manual de los turnos.
- Dificultad para consultar el historial de servicios realizados.
- Retrasos ocasionados por la sobrecarga de trabajo.
- Información dispersa entre llamadas, mensajes y registros físicos.
- Falta de un control centralizado sobre los servicios técnicos.

Estas situaciones afectan tanto a los propietarios de los talleres como a los clientes, quienes muchas veces desconocen cuándo será atendido su vehículo o cuál es el avance del trabajo solicitado.

Como alternativa, se propone el desarrollo de una API REST que permita administrar de forma centralizada la información relacionada con los turnos y los servicios técnicos, facilitando el acceso a la información y mejorando la organización interna del taller.

2.3. Objetivos del proyecto

2.3.1. Objetivo general

Desarrollar una API REST para gestionar los turnos y los servicios técnicos de talleres mecánicos pequeños de la ciudad de Manta, utilizando tecnologías modernas para mejorar la organización de la información y optimizar el proceso de atención a los clientes.

2.3.2. Objetivos específicos

- Analizar la forma en que los talleres administran actualmente sus actividades.
- Diseñar una estructura de datos adecuada para representar la información del sistema.
- Implementar los servicios REST necesarios para administrar los módulos del proyecto.
- Incorporar mecanismos de autenticación que protejan el acceso a la información.
- Facilitar futuras ampliaciones mediante una arquitectura organizada.

2.4. Usuarios del sistema

Cliente

Corresponde a la persona que solicita un servicio para su vehículo.

Sus principales necesidades son:

- Solicitar un turno.
- Conocer el estado del servicio.
- Consultar información relacionada con su vehículo.
- Evitar tiempos de espera innecesarios.

Administrador del taller

Es el responsable de gestionar la información del taller.

Entre sus funciones principales se encuentran:

- Registrar nuevos turnos.
- Administrar los servicios técnicos.
- Consultar la información almacenada.
- Actualizar el estado de las reparaciones.
- Mantener organizada la información de los clientes.

2.5. Investigación realizada

Con el propósito de validar la problemática se realizaron entrevistas a personas relacionadas con talleres mecánicos de la ciudad de Manta.

Las preguntas estuvieron orientadas a conocer:

- Cómo organizan actualmente los turnos.
- Qué herramientas utilizan diariamente.
- Cuáles son los principales inconvenientes durante la atención.
- Qué aspectos consideran que podrían mejorar mediante un sistema informático.

Las respuestas obtenidas permitieron confirmar que la mayoría de los establecimientos pequeños todavía utilizan procedimientos manuales para administrar la información, generando problemas de organización cuando aumenta el número de clientes.

2.6. Entrevistas realizadas

Con el propósito de conocer de primera mano cómo se administra la atención en talleres mecánicos de la ciudad de Manta, se realizaron entrevistas a personas relacionadas con este tipo de negocios y a usuarios que han utilizado sus servicios.

Entrevista 1

Nombre: Javier Bailón Vélez

Edad: 42 años

Ocupación: Encargado de taller mecánico (Manta Auto)

Ubicación: San Agustín, Manta

Respuesta destacada

"Hasta la vez no hemos tenido mayores inconvenientes porque atendemos conforme llegan los clientes. Cuando estamos ocupados les indicamos una fecha aproximada y registramos la información en Excel. Después les avisamos por WhatsApp o mediante una llamada."

Hallazgos

El taller utiliza hojas de cálculo para registrar algunos trabajos.

La comunicación con los clientes se realiza principalmente por WhatsApp y llamadas telefónicas.

No existe un sistema que permita consultar la disponibilidad de los turnos de forma automática.

El historial de servicios depende de archivos elaborados manualmente.

Entrevista 2

Nombre: Luis Mendoza

Edad: 34 años

Ocupación: Conductor profesional

Ubicación: Manta

Respuesta destacada

"En algunas ocasiones he tenido que esperar varias horas porque el taller recibió otros vehículos antes del mío. Muchas veces uno no sabe cuánto tiempo realmente va a tardar el trabajo."

Hallazgos

Los clientes desconocen el tiempo real de atención.

Existe poca comunicación sobre el estado de los trabajos.

No siempre se respeta el orden de llegada cuando aparecen trabajos urgentes.

2.6.1. Entrevista en video

Como evidencia del levantamiento de información se realizó una entrevista en video con uno de los responsables de un taller mecánico de la ciudad de Manta.

En el video se explica:

El procedimiento utilizado actualmente para registrar vehículos.

La forma en que organizan los trabajos diarios.

Las dificultades para mantener organizada la información cuando existen muchos vehículos pendientes.

Duración aproximada: 2 minutos.

2.7. Análisis de soluciones existentes

Durante la investigación se identificó que la mayoría de los talleres pequeños utilizan herramientas sencillas para registrar la información de sus clientes.

Herramienta	Uso principal	Limitación
WhatsApp	Coordinar turnos	Conversaciones difíciles de organizar
Excel	Registrar clientes y trabajos	No existe acceso concurrente ni historial estructurado
Cuadernos	Anotar reparaciones	Riesgo de pérdida de información
Llamadas telefónicas	Confirmar horarios	No queda evidencia del registro

Aunque estas herramientas permiten mantener un control básico, presentan limitaciones cuando el volumen de clientes aumenta o cuando se necesita consultar información histórica.

Entre las principales limitaciones encontradas se destacan:

- Información distribuida en diferentes medios.
- Dificultad para consultar trabajos anteriores.
- Organización manual de los turnos.
- Falta de control sobre el estado de los servicios.
- Riesgo de pérdida de información.

2.8. Propuesta de solución

Como respuesta a la problemática identificada se desarrolló una API REST orientada a la administración de talleres mecánicos.

El sistema fue implementado utilizando el lenguaje de programación Go y una arquitectura organizada por módulos, permitiendo administrar la información mediante servicios HTTP.

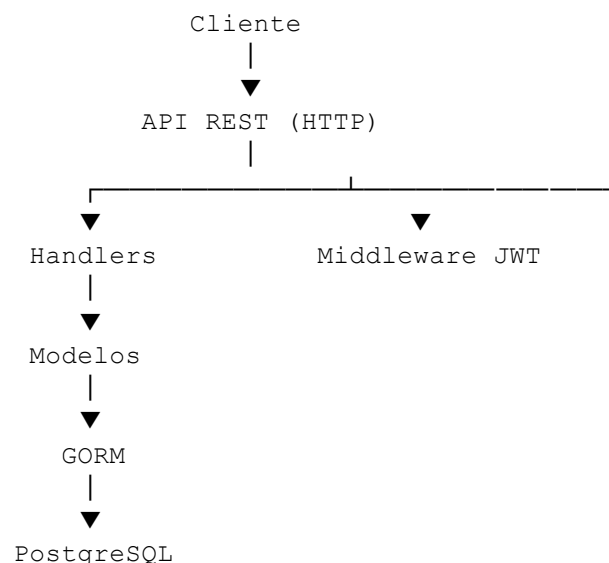
La solución incorpora funcionalidades para:

- Registrar turnos de atención.
- Gestionar los servicios técnicos realizados.
- Administrar clientes y vehículos.
- Registrar mecánicos y especialidades.
- Controlar el acceso mediante autenticación JWT.
- Almacenar la información utilizando una base de datos relacional.

El sistema fue diseñado pensando en talleres pequeños y medianos que requieren una solución sencilla, organizada y de fácil mantenimiento.

2.9. Arquitectura general del sistema

La aplicación sigue una arquitectura en capas, separando la lógica del negocio de la manipulación de datos y de las rutas expuestas por la API.



Esta organización facilita el mantenimiento del proyecto y permite agregar nuevas funcionalidades sin modificar la estructura principal.

2.10. Módulos implementados

El proyecto fue dividido en módulos para facilitar el desarrollo y la organización del código.

2.10.1. Módulo de Gestión de Turnos

Este módulo permite registrar y administrar la información relacionada con la atención de los vehículos.

Entre sus funciones principales se encuentran:

- Registrar nuevos turnos.
- Consultar los turnos existentes.
- Actualizar información.
- Eliminar registros.
- Consultar el estado de un turno.

2.10.2. Módulo de Servicios Técnicos

Este módulo registra la información relacionada con los trabajos realizados sobre cada vehículo.

Permite:

- Registrar servicios.
- Consultar servicios realizados.
- Modificar información.
- Eliminar registros.
- Asociar un servicio con el turno correspondiente.

2.11. Entidades del sistema

El proyecto actualmente utiliza las siguientes entidades principales:

- Usuario
- Cliente
- Vehículo
- Mecánico
- Especialidad
- Turno
- Servicio

Cada entidad representa un componente específico del funcionamiento del taller y mantiene relaciones con las demás mediante identificadores.

2.12. Regla de negocio principal

La funcionalidad principal del sistema consiste en administrar la asignación de turnos considerando la disponibilidad del taller.

Antes de registrar un nuevo turno se verifica que el horario solicitado no genere conflictos con otros registros existentes y que exista disponibilidad para atender el vehículo.

De esta forma se evita la sobrecarga de trabajo y se mantiene una mejor organización de las actividades diarias.

2.13. Conclusión del Hito 1

El proceso de investigación permitió identificar que los talleres mecánicos pequeños de la ciudad de Manta todavía presentan dificultades para administrar la información relacionada con sus clientes y los servicios ofrecidos.

Las entrevistas realizadas evidenciaron la necesidad de contar con una herramienta informática que facilite la organización de los turnos y el registro de los servicios técnicos.

Con base en esta información se definió la estructura inicial del sistema y los módulos que serían implementados durante el desarrollo del proyecto, estableciendo las bases para la construcción de una API REST segura, organizada y preparada para futuras ampliaciones.

3. HITO 2

Desarrollo de la API REST

3.1. Introducción

Una vez identificada la problemática y definidos los requerimientos del sistema, se inició la fase de desarrollo de la API REST. Durante este hito se implementó la estructura principal del proyecto, organizando el código en módulos independientes y aplicando una arquitectura que facilitara el mantenimiento y la incorporación de nuevas funcionalidades.

Para el desarrollo se utilizó el lenguaje de programación Go debido a su buen rendimiento en aplicaciones web y su facilidad para construir servicios REST.

Además, se empleó el framework Chi Router para gestionar las rutas de la aplicación y GORM como herramienta de acceso a la base de datos.

El objetivo principal de este hito fue construir una API funcional que permitiera administrar la información de los talleres mediante operaciones CRUD sobre las diferentes entidades del sistema.

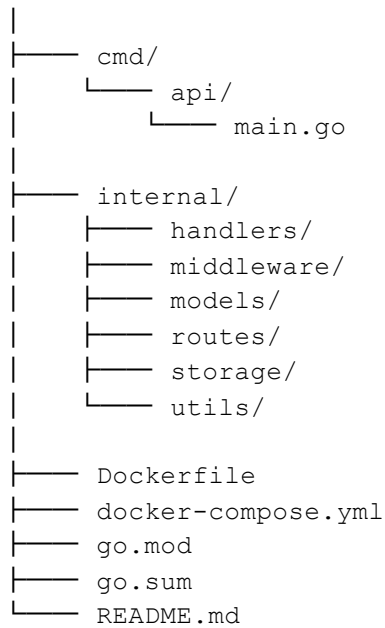
3.2. Tecnologías utilizadas

Durante el desarrollo del proyecto se emplearon las siguientes herramientas:

Tecnología	Descripción
Go	Lenguaje principal utilizado para desarrollar la API.
Chi Router	Framework encargado de administrar las rutas HTTP.
GORM	ORM utilizado para interactuar con la base de datos.
PostgreSQL	Sistema gestor de base de datos relacional.
JWT	Autenticación mediante tokens.
Docker	Contenerización del proyecto.
Docker Compose	Orquestación de los servicios necesarios para ejecutar la aplicación.
Git	Control de versiones.
GitHub	Repositorio remoto del proyecto.
GitHub Actions	Automatización de pruebas mediante integración continua.

3.3. Organización del proyecto

Con el propósito de mantener el código organizado, el proyecto fue dividido en diferentes carpetas, cada una con una responsabilidad específica.

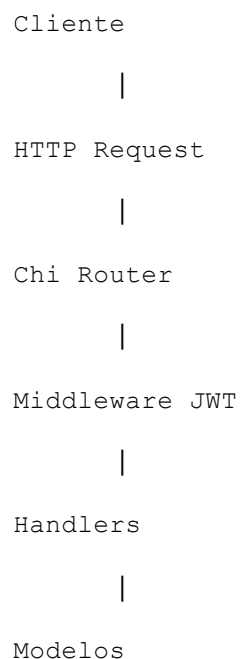


Esta organización permite separar la lógica de negocio, los modelos de datos, la configuración de rutas y los componentes de autenticación, facilitando el mantenimiento del sistema.

3.4. Arquitectura implementada

El sistema fue construido siguiendo una arquitectura por capas.

Cada petición enviada por el cliente atraviesa diferentes componentes antes de acceder a la base de datos.



|

GORM

|

PostgreSQL

Esta arquitectura permite mantener una separación clara entre las responsabilidades de cada componente y facilita futuras ampliaciones del sistema.

3.5. Módulos desarrollados

El sistema fue dividido en dos módulos principales de acuerdo con la distribución del trabajo entre los integrantes del grupo.

3.5.1. Módulo de Gestión de Turnos

Este módulo permite registrar la información relacionada con la atención de los vehículos.

Entre las operaciones implementadas se encuentran:

- Registrar un nuevo turno.
- Consultar todos los turnos registrados.
- Buscar un turno específico.
- Modificar la información de un turno.
- Eliminar un turno.

La administración de los turnos constituye uno de los procesos principales del sistema, ya que permite organizar la atención de los vehículos y controlar la disponibilidad del taller.

3.5.2. Módulo de Servicios Técnicos

Este módulo administra la información correspondiente a los trabajos realizados sobre cada vehículo.

Las funcionalidades implementadas permiten:

- Registrar nuevos servicios.
- Consultar servicios existentes.
- Actualizar información.
- Eliminar registros.
- Relacionar cada servicio con un turno previamente registrado.

Este módulo facilita la consulta del historial de trabajos realizados y permite mantener organizada la información técnica del taller.

3.6. Modelos implementados

Aunque el proyecto fue dividido en dos módulos principales, la base de datos incorpora varias entidades que permiten representar adecuadamente el funcionamiento del sistema.

Las entidades implementadas son:

- Usuario
- Cliente
- Vehículo
- Mecánico
- Especialidad
- Turno
- Servicio

Cada una de ellas fue representada mediante estructuras (`struct`) en Go, permitiendo que GORM realizara automáticamente el mapeo hacia las tablas correspondientes en PostgreSQL.

Las relaciones entre estas entidades facilitan la organización de la información y evitan la duplicidad de datos.

3.7. Operaciones CRUD

Para cada uno de los módulos principales se implementaron las operaciones CRUD (Create, Read, Update y Delete), permitiendo administrar completamente la información almacenada en el sistema.

Las operaciones disponibles son:

Gestión de Turnos

Método	Función
POST	Registrar un turno
GET	Consultar todos los turnos
GET	Consultar un turno por ID

Método	Función
PUT	Actualizar un turno
DELETE	Eliminar un turno

Servicios Técnicos

Método	Función
POST	Registrar un servicio
GET	Consultar todos los servicios
GET	Consultar un servicio por ID
PUT	Actualizar un servicio
DELETE	Eliminar un servicio

Cada endpoint devuelve respuestas en formato JSON y utiliza los códigos de estado HTTP correspondientes según el resultado de la operación.

3.8. Validaciones implementadas

Con el propósito de garantizar la integridad de la información, durante el desarrollo se incorporaron validaciones básicas antes de registrar o modificar los datos.

Entre las principales validaciones implementadas se encuentran:

- Verificación de campos obligatorios.
- Validación del formato de los datos recibidos.
- Comprobación de registros existentes antes de actualizar o eliminar información.
- Manejo de errores cuando un recurso solicitado no existe.

Estas validaciones permiten evitar inconsistencias y mejoran la confiabilidad de la API.

3.9. Pruebas realizadas

Para comprobar el funcionamiento de los servicios implementados se utilizó Postman.

Durante las pruebas se verificó el correcto funcionamiento de los diferentes endpoints mediante solicitudes HTTP utilizando los métodos GET, POST, PUT y DELETE.

Las respuestas obtenidas fueron analizadas verificando:

- Código de estado HTTP.
- Información retornada en formato JSON.
- Registro correcto de los datos.
- Actualización de la información.
- Eliminación de registros.

Figura 1. Pruebas realizadas con Postman. (Insertar captura de la colección de Postman ejecutando los endpoints del módulo de Turnos y Servicios).

3.10. Resultados obtenidos

Al finalizar el desarrollo de este hito se obtuvo una API funcional capaz de administrar la información relacionada con los turnos y los servicios técnicos del taller.

Las operaciones CRUD respondieron correctamente durante las pruebas realizadas, permitiendo registrar, consultar, modificar y eliminar información mediante solicitudes HTTP.

La estructura modular implementada facilitó la organización del código y permitió mantener una separación clara entre los diferentes componentes del sistema.

3.11. Conclusiones del Hito 2

Durante esta etapa se logró desarrollar la estructura principal de la API REST, implementando los módulos necesarios para administrar la información del sistema.

El uso de Go junto con Chi Router permitió construir servicios rápidos y organizados, mientras que la integración con GORM simplificó el acceso a la base de datos.

Asimismo, las pruebas realizadas demostraron que los endpoints implementados cumplen correctamente con las operaciones CRUD requeridas para el funcionamiento del sistema.

4. HITO 3

Integración, seguridad y despliegue del sistema

4.1. Introducción

Durante el desarrollo del tercer hito se incorporaron las funcionalidades necesarias para completar la implementación de la API REST. En esta etapa se trabajó principalmente en la autenticación de usuarios, la protección de rutas, la persistencia de datos mediante PostgreSQL, la contenerización del proyecto utilizando Docker y la automatización de procesos mediante GitHub Actions.

Estas mejoras permitieron obtener una aplicación más segura, organizada y preparada para ser ejecutada en diferentes entornos sin depender de configuraciones específicas de cada equipo.

4.2. Integración de la base de datos

Una vez finalizada la estructura principal de la API, se reemplazó el almacenamiento temporal utilizado durante las primeras pruebas por una base de datos PostgreSQL.

Para facilitar la interacción con la base de datos se utilizó **GORM**, un ORM para Go que permite mapear automáticamente las estructuras del lenguaje hacia tablas relacionales.

Entre las ventajas obtenidas con GORM se destacan:

- Creación automática de tablas.
- Definición sencilla de relaciones entre entidades.
- Consultas más legibles.
- Menor cantidad de código SQL.
- Facilidad para realizar operaciones CRUD.

Gracias a esta herramienta fue posible mantener el código más organizado y reducir el tiempo de desarrollo.

4.3. Modelado de la base de datos

La base de datos quedó conformada por las siguientes entidades:

Entidad	Descripción
Usuario	Almacena la información necesaria para la autenticación del sistema.
Cliente	Registra la información de los propietarios de los vehículos.
Vehículo	Contiene los datos de los vehículos registrados.
Mecánico	Registra la información del personal técnico del taller.
Especialidad	Permite clasificar las habilidades de los mecánicos.
Turno	Administra la programación de la atención de los vehículos.
Servicio	Registra los trabajos realizados durante cada reparación.

Las relaciones entre estas entidades permiten mantener la información organizada y evitar redundancia de datos.

4.4. Autenticación mediante JWT

Uno de los principales objetivos de este hito fue proteger el acceso a los servicios de la API.

Para ello se implementó un sistema de autenticación basado en **JSON Web Token (JWT)**.

El proceso de autenticación funciona de la siguiente manera:

- El usuario envía sus credenciales al servidor.
- El sistema verifica que la información sea correcta.
- Si las credenciales son válidas, se genera un token JWT.
- El cliente almacena el token.
- En las siguientes solicitudes el token es enviado mediante el encabezado **Authorization**.
- El servidor valida el token antes de permitir el acceso al recurso solicitado.

Este mecanismo evita que usuarios no autorizados puedan acceder a información protegida.

4.5. Middleware de autenticación

Para validar los tokens generados se implementó un middleware encargado de interceptar todas las solicitudes dirigidas a las rutas protegidas.

El middleware realiza las siguientes acciones:

- Verifica que exista un encabezado Authorization.
- Comprueba que el token tenga el formato correcto.
- Valida la firma del token.
- Comprueba que el token no haya expirado.
- Permite continuar la solicitud únicamente cuando la autenticación es válida.

En caso contrario, la API devuelve un código de estado **401 Unauthorized**.

4.6. Organización de las rutas

Con el crecimiento del proyecto fue necesario separar las rutas en diferentes grupos para facilitar el mantenimiento del código.

La organización utilizada permite distinguir claramente las rutas públicas de aquellas que requieren autenticación.

Ejemplo:

```
/api/auth/login  
/api/auth/register  
  
/api/clientes  
/api/vehiculos  
/api/mecanicos  
/api/especialidades  
/api/turnos  
/api/servicios
```

Esta estructura mejora la legibilidad del proyecto y facilita la incorporación de nuevos módulos.

4.7. Contenerización mediante Docker

Con el objetivo de simplificar la ejecución del proyecto en diferentes equipos se implementó Docker.

La aplicación fue configurada para ejecutarse dentro de un contenedor, evitando problemas relacionados con versiones del lenguaje, dependencias o configuraciones locales.

Entre las ventajas obtenidas se encuentran:

- Instalación más sencilla.
- Entorno de ejecución uniforme.
- Mayor facilidad para compartir el proyecto.
- Despliegue más rápido.

Figura 2. Contenedor de la API ejecutándose en Docker Desktop. *(Insertar captura de Docker Desktop con el contenedor en ejecución).*

4.8. Docker Compose

Además del contenedor principal de la aplicación, se utilizó Docker Compose para administrar conjuntamente la API y la base de datos PostgreSQL.

Gracias a esta herramienta ambos servicios pueden iniciarse utilizando un único comando.

Esta configuración facilita el desarrollo colaborativo y reduce el tiempo necesario para preparar el entorno de trabajo.

Figura 3. Ejecución del proyecto mediante Docker Compose. *(Insertar captura de la terminal mostrando docker compose up o los contenedores activos).*

4.9. Integración continua mediante GitHub Actions

Como parte del proceso de desarrollo se configuró un flujo básico de integración continua utilizando GitHub Actions.

Cada vez que se realiza un cambio en el repositorio, GitHub ejecuta automáticamente una serie de verificaciones para comprobar que el proyecto pueda compilar correctamente.

Entre las tareas realizadas se encuentran:

- Descarga de dependencias.
- Compilación del proyecto.
- Ejecución de pruebas configuradas.
- Verificación de errores durante la construcción.

La integración continua ayuda a detectar errores antes de integrar nuevos cambios al proyecto.

Figura 4. Ejecución del flujo de GitHub Actions.
(Insertar captura del workflow ejecutado correctamente).

4.10. Flujo general del sistema

El funcionamiento general de la aplicación puede resumirse en el siguiente proceso:

Cliente



Solicitud HTTP



Chi Router



Middleware JWT



Handler correspondiente



Modelo



GORM

↓

PostgreSQL

↓

Respuesta JSON

Este flujo representa el recorrido que sigue una petición desde que es enviada por el cliente hasta que se obtiene la respuesta correspondiente.

4.11. Resultados obtenidos

Al finalizar el desarrollo del proyecto se obtuvo una API REST completamente funcional capaz de administrar la información relacionada con clientes, vehículos, mecánicos, especialidades, turnos y servicios técnicos.

La incorporación de JWT permitió mejorar la seguridad del sistema, mientras que Docker facilitó la ejecución del proyecto en distintos equipos sin modificar la configuración del entorno.

Asimismo, el uso de PostgreSQL y GORM permitió mantener la información organizada y persistente, ofreciendo una solución más robusta que el almacenamiento temporal utilizado durante las primeras etapas del desarrollo.

5. Conclusiones generales

El desarrollo de este proyecto permitió aplicar de manera práctica los conocimientos adquiridos durante la asignatura Aplicaciones Web II.

La implementación de una API REST utilizando Go representó una experiencia importante para comprender la organización de aplicaciones backend, el diseño de servicios web y la integración de diferentes tecnologías dentro de un mismo proyecto.

Además de implementar las operaciones CRUD, se incorporaron mecanismos de autenticación, persistencia de datos y herramientas de despliegue que mejoran la seguridad y la mantenibilidad del sistema.

Finalmente, el proyecto demuestra que es posible desarrollar soluciones tecnológicas capaces de contribuir a la organización de pequeños talleres mecánicos, optimizando el manejo de la información y facilitando el control de los procesos diarios.

6. Recomendaciones

Para futuras versiones del sistema se recomienda:

- Incorporar un módulo de notificaciones para informar a los clientes sobre el estado de sus vehículos.
- Desarrollar una interfaz web que consuma la API y facilite la interacción con los usuarios.
- Implementar control de roles con diferentes niveles de permisos.
- Agregar reportes estadísticos sobre los servicios realizados y la carga de trabajo del taller.
- Realizar pruebas automatizadas con mayor cobertura para garantizar la estabilidad del sistema.